Fullstack labsheet 6

Name: Vishavjeet Baraily

CUID: CU24250187

Q1. Server Setup & Building the REST API

(a) Setup Commands & .env File

Terminal setup inside project root:

Bash

```
mkdir api-lab && cd api-lab
mkdir server client
cd server
npm init -y
npm install express cors dotenv
npm install --save-dev nodemon
```

server/.env:

Code snippet

```
PORT=5000
API_KEY=mysecretkey123
```

server/package.json (updated scripts section):

JSON

```
"scripts": {
  "dev": "nodemon index.js"
}
```

(b) & (c) Express Server, In-Memory CRUD Routes & Middleware (server/index.js)

JavaScript

```
const express = require('express');
const cors = require('cors');
require('dotenv').config();
const app = express();
const PORT = process.env.PORT || 5000;
app.use(cors());
app.use(express.json());
```

```
// Global Logger Middleware
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
  next();
});
// Auth Middleware for protected routes
const requireApiKey = (req, res, next) => {
const apiKey = req.headers['x-api-key'];
  if (!apiKey || apiKey !== process.env.API_KEY) {
    return res.status(401).json({ error: 'Unauthorized: Invalid or missing API key' });  }
  next();};
// In-Memory Data
let tasks = [
  { id: 1, title: 'Setup Server', completed: true },
  { id: 2, title: 'Build API Routes', completed: false },
  { id: 3, title: 'Add Middleware', completed: false },
  { id: 4, title: 'Configure CORS', completed: false },
  { id: 5, title: 'Connect Client', completed: false }
];
// GET all tasks
app.get('/api/tasks', (req, res) => {
res.json(tasks);
});
// GET single task by ID
app.get('/api/tasks/:id', (req, res) => {
  const task = tasks.find(t => t.id === parseInt(req.params.id));
  if (!task) return res.status(404).json({ error: 'Task not found' });
  res.json(task);
});
```

```
// POST new task (Protected)
app.post('/api/tasks', requireApiKey, (req, res) => {
const newTask = {
    id: tasks.length ? Math.max(...tasks.map(t => t.id)) + 1 : 1,
    title: req.body.title || 'Untitled Task',
    completed: false
  };
  tasks.push(newTask);
  res.status(201).json(newTask);
});
// PUT update task (Protected)
app.put('/api/tasks/:id', requireApiKey, (req, res) => {
  const task = tasks.find(t => t.id === parseInt(req.params.id));
  if (!task) return res.status(404).json({ error: 'Task not found' });
  if (req.body.title !== undefined) task.title = req.body.title;
  if (req.body.completed !== undefined) task.completed = req.body.completed;
  res.json(task);
});
// DELETE task (Protected)
app.delete('/api/tasks/:id', requireApiKey, (req, res) => {
  const index = tasks.findIndex(t => t.id === parseInt(req.params.id));
  if (index === -1) return res.status(404).json({ error: 'Task not found' });
  const deletedTask = tasks.splice(index, 1);
  res.json({ message: 'Task deleted successfully', task: deletedTask[0] });
});
// Q2(c) 404 Unmatched Route Handler
app.use((req, res, next) => {
  res.status(404).json({ error: 'Route not found' });
});
// Q2(c) Centralized Error Handling Middleware
app.use((err, req, res, next) => {
```

```
  console.error(err.stack);
  res.status(500).json({ error: err.message || 'Internal Server Error' });
});
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Q2. Securing, Testing & Error-Handling the API

(a) CORS Testing Note

- **CORS Disabled Error:** Access to fetch at 'http://localhost:5000/api/tasks' from origin 'http://localhost:3000' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.
- **Fix:** Re-enabling app.use(cors()) allows cross-origin requests by adding appropriate HTTP headers.

(b) Postman Setup Summary

- **Collection:** API Lab
- **GET /api/tasks:** Works with 200 OK (No API key needed).
- **POST /api/tasks (Without Key):** Returns 401 Unauthorized.
- **POST /api/tasks (With Header x-api-key: mysecretkey123):** Returns 201 Created.

Q3. Consuming the API — Plain JavaScript Client

client/index.html

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Tasks - Plain JS</title>
</head>
<body>
  <h2>Task List (Vanilla JS)</h2>
  <div id="loading">Loading...</div>
  <div id="error" style="color: red;"></div>
  <ul id="task-list"></ul>
```

```
  <script src="script.js"></script>
</body>
</html>
```

client/script.js

JavaScript

```
const loadingEl = document.getElementById('loading');
const errorEl = document.getElementById('error');
const listEl = document.getElementById('task-list')
fetch('http://localhost:5000/api/tasks')
  .then(res => {
    if (!res.ok) throw new Error(`HTTP error! Status: ${res.status}`);
    return res.json();
  })
  .then(data => {
    loadingEl.style.display = 'none';
    data.forEach(task => {
      const li = document.createElement('li');
      li.textContent = task.title;
      listEl.appendChild(li) });  })
  .catch(err => {
    loadingEl.style.display = 'none';
    errorEl.textContent = 'Failed to load tasks: ' + err.message;
  });
```

Q4 & Q5. Consuming the API — React Client & Full-Stack Integration

Setup Commands (client-react):

Bash

```
npm create vite@latest client-react----- template react
cd client-react
npm install axios
```

.env File (client-react/.env):

Code snippet

```
VITE_API_URL=http://localhost:5000/api
```

client-react/src/TaskList.jsx

JavaScript

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
const API_URL = import.meta.env.VITE_API_URL || 'http://localhost:5000/api';
const API_KEY = 'mysecretkey123'; // Matches server .env
export default function TaskList() {
  const [tasks, setTasks] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [newTitle, setNewTitle] = useState('');
  // Fetch tasks using Axios
  const fetchTasks = async () => {
  setLoading(true);
  setError(null);
    try {
      // Axios automatically parses JSON and throws on HTTP errors (e.g. 404, 500)
      // Unlike fetch(), no manual `if (!res.ok)` check is required.
      const res = await axios.get(`${API_URL}/tasks`);
      setTasks(res.data);
    } catch (err) {
      setError(err.response?.data?.error || err.message);
    } finally {
      setLoading(false);
    }
  };
  useEffect(() => {
    fetchTasks();
  }, []);
```

```
// Create Task (POST)
const handleAddTask = async (e) => {
  e.preventDefault();
  if (!newTitle.trim()) return;
  try {
    const res = await axios.post(
      `${API_URL}/tasks`,
      { title: newTitle },
      { headers: { 'x-api-key': API_KEY } }
    );
    setTasks([...tasks, res.data]);
    setNewTitle('');
  } catch (err) {
    setError(err.response?.data?.error || err.message);
  }
};
 // Delete Task (DELETE)
const handleDeleteTask = async (id) => {
  try {
    await axios.delete(`${API_URL}/tasks/${id}`, {
      headers: { 'x-api-key': API_KEY }
    });
    setTasks(tasks.filter(t => t.id !== id));
  } catch (err) {
    setError(err.response?.data?.error || err.message);
  }
};
if (loading) return <div>Loading...</div>;
return (
  <div style={{ padding: '20px', fontFamily: 'sans-serif' }}>
```

```
    <h2>Task Manager Mini App</h2>
    {error && <div style={{ color: 'red', marginBottom: '10px' }}>Error: {error}</div>}
    <form onSubmit={handleAddTask} style={{ marginBottom: '15px' }}>
     <input
      type="text"
      value={newTitle}
      onChange={(e) => setNewTitle(e.target.value)}
      placeholder="New Task Title"
     />
     <button type="submit" style={{ marginLeft: '8px' }}>Add Task</button>
    </form>
    <ul>
     {tasks.map(task => (
      <li key={task.id} style={{ marginBottom: '8px' }}>
       {task.title}
       <button
        onClick={() => handleDeleteTask(task.id)}
        style={{ marginLeft: '10px', color: 'red' }}
       >
        Delete
       </button>
      </li>
     ))}
    </ul>
    {/* Button to test error handling using an invalid ID */}
    <button onClick={() => handleDeleteTask(9999)} style={{ marginTop: '10px' }}>
     Test Delete Invalid ID (9999)
    </button>
   </div>
  );
}
```